

NAINA OS

SYSTEM ARCHITECTURE & ENGINEERING SPECIFICATION

Next-Generation Open-Source AI Operating System & Digital Human Platform

DOCUMENT METADATA

Document Identifier:	NOS-ENG-SPEC-2026-V1.0
Specification Version:	1.0.0 (Production Engineering Spec)
Document Status:	APPROVED -- Open Standard
Date of Issue:	July 31, 2026
Author Group:	Core Systems Architecture Group
Target Audience:	Systems Engineers, AI Architects, HCI Leads
Core Paradigm:	Human First * Memory First * Local First * Agent First

This document contains formal engineering specifications for the NAINA OS platform. All designs adhere to local-first, privacy-first, and event-driven micro-kernel standards.

NAINA OS -- System Architecture & Engineering Specification

Document Revision History

Date	Revision	Author	Description of Changes
---	---	---	---
2026-07-31	v1.0.0	Core Systems Architecture Group	Initial Release of Complete NAINA OS Eng
2026-07-15	v0.9.0	AI Systems & Agent Engineering Te	Internal RFC Draft for Dual-Persona Cogn
2026-06-01	v0.1.0	Kernel Systems Group	Initial Architectural Blueprint & Bounda

1. Executive Summary & Purpose

1.1 Purpose

This document provides the definitive, comprehensive engineering specification for NAINA OS -- an open-source, local-first, memory-centric AI Operating System and Digital Human Platform. Designed to supersede traditional single-purpose chatbot wrappers, NAINA OS operates as an autonomous, multimodal, multi-agent operating system capable of native hardware desktop control, mobile orchestration, real-time voice synthesis, structured procedural memory management, and cross-platform task automation.

1.2 Scope

This specification governs all core software layers of NAINA OS, including:

- Kernel & Event Bus Architecture: Asynchronous, event-driven decoupled IPC.
- Dual-Persona Cognitive Core: Psychological and functional partitioning between NAINA (Companion/Planner) and CENANI (Systems/Execution Engine).
- Multimodal Input/Output Subsystem: Low-latency voice processing (Whisper, Piper, Kokoro) and spatial vision input (Qwen-VL).
- Memory Engine Architecture: Hybrid multi-tiered memory combining PostgreSQL with pgvector, ChromaDB/Qdrant, and local Obsidian Markdown file sync.
- Multi-Agent & Tool Orchestration: Model Context Protocol (MCP) host execution, Playwright browser automation, Android Debug Bridge (ADB) mobile control, and Docker container sandboxing.
- Security & Permission Model: Capability-based security, local air-gap sandboxing, explicit user permission gating, and audit logging.

2. Core Architectural Principles & Trade-off Rationale

2.1 Foundational Principles

NAINA OS is built on nine non-negotiable engineering mandates:

CORE PRINCIPLES MATRIX		
1. Human First	4. Privacy First	7. Agent First
2. Memory First	5. Open Source	8. Obsidian First
3. Local First	6. Plugin First	9. Zero Cost Default

- 1. Human First: Interfaces prioritize natural interaction (voice, vision, high-EQ digital avatars) over rigid CLI/GUI paradigms.
- 2. Memory First: Context is preserved continuously. The system never starts from a blank slate; every interaction updates the persistent knowledge graph.
- 3. Local First: All inference, embedding generation, storage, and orchestration run locally on user hardware by default.
- 4. Privacy First: Zero telemetry, zero unencrypted cloud syncing, and zero remote data leakage without explicit user permission.
- 5. Open Source: Built exclusively on open standards, permissive licenses, and accessible codebases.
- 6. Plugin First & Agent First: Functional capabilities are implemented as isolated, dynamic Model Context Protocol (MCP) agents rather than monolithic hardcoded features.
- 7. Obsidian First: Plaintext Markdown with bidirectional linking serves as the primary human-readable knowledge vault and procedural memory mirror.
- 8. Modular Architecture & Micro-Kernel Isolation: Nothing talks directly to anything else. All inter-module communication flows through a structured, typed Event Bus.
- 9. Zero Cost: Optimized to run efficiently on standard consumer hardware (e.g., Apple Silicon M-series or NVIDIA RTX mid-tier GPUs) using quantized open-weights models (Ollama, Qwen, Whisper, Kokoro).

2.2 Engineering Trade-offs & Rationale

[!NOTE]

Architectural Decision Rationale: Event-Driven Micro-Kernel vs. Monolithic Process

**Decision:* Implement an asynchronous Event Bus decoupled architecture over monolithic in-process method invocation.*

**Why:* Monolithic agent architectures lead to race conditions, untraceable state mutations, and system crashes when third-party tools fail. An event-driven bus allows dynamic hot-swapping of sub-agents, deterministic audit logging, and isolated process recovery.*

**Trade-off:* Introduces serialization overhead (~1.2ms per IPC message) and requires strict schema versioning.*

[!IMPORTANT]

Architectural Decision Rationale: Obsidian Markdown Vault as Primary Human Memory Mirror

**Decision:* Synchronize all long-term vector/graph memory updates down to standard Markdown files in an Obsidian-compatible structure.*

**Why:* Vector databases are opaque "black boxes." If a database corrupts or a model embedding schema shifts, the user's Second Brain is lost. Markdown files ensure user data ownership, portability, direct manual editing, and git version control.*

**Trade-off:* Syncing relational/vector states to disk file structures requires file-system watchers and conflict-resolution lock mechanisms.*

3. High-Level System Architecture

3.1 Architectural Rule #1

"Nothing talks directly to anything else. Everything communicates through a defined architecture."

No module (Voice, GUI, LLM Router, Memory, Tool Executor) is permitted to call internal methods of another module directly. All interactions are converted into typed System Events published to the central NAINA OS Event Bus.

3.2 System Topology Diagram

```

graph TD
    %% User Inputs
    subgraph Layer_1_and_2 [Layer 1 & 2: Interface & Multimodal Processing]
        User((User)) <-->|Voice / Audio| AudioIO[Whisper STT / Piper-Kokoro TTS]
        User <-->|GUI / Touch| Frontend[Next.js 14 / React / Framer Motion]
        User <-->|Camera / Vision| VisionIO[Qwen-VL Vision Engine]
    end

    %% Event Bus & Router
    subgraph Kernel_Core [Layer 3, 4 & 9: Kernel, Routing & Security]
        AudioIO -->|EVT_AUDIO_INPUT| EventBus((NAINA OS Core Event Bus))
        Frontend -->|EVT_USER_COMMAND| EventBus
        VisionIO -->|EVT_FRAME_CAPTURED| EventBus

        EventBus <--> SecurityEngine[Layer 9: Security Sandbox & RBAC Guardrails]
        SecurityEngine <--> Router[Layer 4: AI Orchestrator & Router Kernel]
    end

    %% Dual Cognition
    subgraph Cognition_Layer [Layer 5 & 6: Dual Persona Engine & Memory]
        Router <-->|Intent: Conversation / Planning| NAINA[NAINA Persona: High EQ/IQ Companion]
        Router <-->|Intent: Systems / Automation| CENANI[CENANI Persona: Operations & Engineeri]

        NAINA <--> MemoryEngine[Layer 6: Multi-Tier Memory Engine]
        CENANI <--> MemoryEngine

        MemoryEngine <--> Postgres[(PostgreSQL + pgvector)]
        MemoryEngine <--> VectorDB[(ChromaDB / Qdrant)]
        MemoryEngine <--> ObsidianSync[Obsidian Vault Markdown Mirror]
    end

    %% Execution & Hardware
    subgraph Execution_Layer [Layer 7, 8 & 10: Agent Orchestration & Execution]
        CENANI -->|Dispatch Execution Task| AgentManager[Layer 7: Agent Manager & Scheduler]

        AgentManager --> MCPHost[Layer 8: MCP Tool Execution Host]

        MCPHost -->|Desktop Control| WinAgent[Windows / Tauri Native Agent]
        MCPHost -->|Mobile Control| AndroidAgent[ADB Android Controller]
        MCPHost -->|Web Control| PlaywrightAgent[Playwright Browser Engine]
        MCPHost -->|Containerized Automation| DockerAgent[Docker Engine Container]
        MCPHost -->|Stream / Media| OBSAgent[OBS Studio Controller]
        MCPHost -->|External API| WorkspaceAgent[GitHub & Google Workspace Adapters]

        WinAgent <--> Hardware[Layer 10: Hardware Abstraction Layer / GPU / NPU]
    end

    %% Event Returns
    MCPHost -->|EVT_TOOL_RESULT| EventBus
    NAINA -->|EVT_SYNTHESIZE_SPEECH| EventBus

```

4. The 10 System Layers

NAINA OS is structured into ten strict, encapsulated layers. Higher layers depend on abstractions provided by lower layers, and all cross-layer operations publish events to the Kernel.

```
+-----+
|                                     |
|          NAINA OS TEN-LAYER STACK          |
|-----+
| LAYER 1: INTERFACE LAYER (Voice UI, Tauri GUI, Canvas, Touch) |
+-----+
| LAYER 2: INPUT PROCESSING & MULTIMODAL FUSION LAYER (STT, Vision, OCR) |
+-----+
| LAYER 3: INTENT & CONTEXT ENGINE (Working Memory, Goal Parser) |
+-----+
| LAYER 4: AI ORCHESTRATOR & ROUTER KERNEL (LLM Cascade, Ollama Bridge) |
+-----+
| LAYER 5: DUAL-PERSONA COGNITION CORE (NAINA vs. CENANI Routing) |
+-----+
| LAYER 6: MEMORY & KNOWLEDGE ENGINE (pgvector, ChromaDB, Obsidian) |
+-----+
| LAYER 7: AGENT & SUBSYSTEM ORCHESTRATION (Agent Manager, Task Queue) |
+-----+
| LAYER 8: ACTION & TOOL EXECUTION LAYER (MCP Host, Playwright, ADB) |
+-----+
| LAYER 9: SECURITY & SANDBOX GUARDRAILS (RBAC, Audit Log, Air-Gap) |
+-----+
| LAYER 10: INFRASTRUCTURE & HARDWARE ABSTRACTION (GPU/NPU, OS Drivers) |
+-----+
```

Layer 1: Interface Layer

- * Responsibility: Manages presentation, visual rendering, voice activation, and user input capture across Desktop (Tauri/Next.js) and Mobile (Android WebView/Native Shell).
- * Key Modules: Voice UI HUD, Floating Avatar Renderer, System Tray Controller, Canvas Workspace.

Layer 2: Input Processing & Multimodal Fusion Layer

- * Responsibility: Converts raw analog sensory data into structured, typed system events.
- * Key Modules:
 - * SpeechToText: Local Faster-Whisper worker (VAD -> Mel Spectrogram -> Tokenizer).
 - * VisionEngine: Screen/Camera frame Sampler utilizing Qwen-VL for spatial element detection.
 - * TouchGestureParser: Multi-touch input normalization for tablet/mobile hosts.

Layer 3: Intent & Context Engine

- * Responsibility: Maintains active operational state, dynamically constructs LLM prompt windows, and performs top-level goal decomposition.
- * Key Modules: Working Memory Manager, Context Truncator, Goal Tree Decomposer.

Layer 4: AI Orchestrator & Router Kernel

- * Responsibility: Determines the optimal LLM execution backend (Local Ollama vs. Cloud APIs) based on capability requirements, latency constraints, and cost profiles.
- * Key Modules:
 - * LLMRouter: Semantic classification of incoming requests.
 - * InferenceBridge: Unified API client connecting to Ollama, vLLM, OpenAI, Anthropic, or Gemini.

Layer 5: Dual-Persona Cognition Core

- * Responsibility: Enforces the psychological separation between emotional companion synthesis and operational system execution.

- * Key Modules: NAINA Cognitive State Machine, CENANI System Protocol Engine.

Layer 6: Memory & Knowledge Engine

- * Responsibility: Maintains persistent multi-tiered storage across episodic, semantic, and procedural domains.
- * Key Modules:
- * VectorStore: pgvector / ChromaDB interface for fast HNSW cosine similarity search.
- * ObsidianVaultSync: Real-time bi-directional synchronization daemon mapping database records to local .md files.

Layer 7: Agent & Subsystem Orchestration

- * Responsibility: Manages agent lifecycles, concurrency limits, dependency graphs, and background task scheduling.
- * Key Modules: Agent Lifecycle Manager, Task Queue (Redis/In-Memory), Concurrency Controller.

Layer 8: Action & Tool Execution Layer

- * Responsibility: Executes discrete side-effecting operations via Model Context Protocol (MCP) servers and OS native bridges.
- * Key Modules: MCP Client Host, Playwright Browser Controller, ADB Bridge, Docker SDK Manager, OBS Studio Socket.

Layer 9: Security & Sandbox Guardrails

- * Responsibility: Inspects all outgoing execution requests against the user capability matrix and logs immutable system audits.
- * Key Modules: RBAC Enforcement Engine, Interactive Approval Middleware (ask_permission), Audit Logger.

Layer 10: Infrastructure & Hardware Abstraction Layer (HAL)

- * Responsibility: Interfaces with underlying host Operating System APIs (Windows Win32, Linux POSIX, macOS Cocoa, Android NDK) and hardware accelerators (NVIDIA CUDA, Apple Metal, ROCm).

5. Digital Humans: Dual-Persona Cognitive Engine (NAINA vs. CENANI)

NAINA OS splits cognitive responsibilities into two distinct AI entities to achieve both high empathy/creativity and uncompromising systems reliability.

DUAL-PERSONA FUNCTIONAL MATRIX	
[NAINA] NAINA (The Companion)	[CENANI] CENANI (The Operator)
* Primary Persona	* Secondary Infrastructure Persona
* High EQ, Empathetic, Playful	* Minimal, Precise, Logical
* Roles: Companion, Researcher, Teacher, Planner, Memory Keeper	* Roles: Systems Engineer, Terminal, Automation, Security, Deployment
* CANNOT execute code/tools directly	* CANNOT initiate emotional dialogue
* Delegates execution to CENANI	* Executes tasks assigned by NAINA

5.1 Enforced Separation Rule

"NAINA never executes. CENANI never decides."

- * NAINA engages with the human user, conducts psychological alignment, formulates high-level strategic plans, updates long-term memory, and synthesizes natural conversational voice responses. When an action is required (e.g., "Deploy this Docker container" or "Scrape this site"), NAINA creates a structured Task Execution Contract and delegates it to CENANI.
- * CENANI acts as the system runtime. CENANI receives the Task Execution Contract, breaks it down into MCP tool calls, executes commands inside isolated sandboxes, captures logs, and returns a verified status report to NAINA. CENANI never generates emotional responses or directly interacts with the primary voice UI.

5.2 Inter-Persona Execution Flow Sequence

```
sequenceDiagram
    autonumber
    actor User
    participant NAINA as [NAINA] NAINA (Companion Engine)
    participant Router as [AI] Router Kernel
    participant Memory as [AI] Memory Engine (Layer 6)
    participant CENANI as [CENANI] CENANI (System Operator)
    participant MCP as [TOOL] MCP Execution Host (Layer 8)
    participant Target as [HOST] Host OS / Target App

    User->>NAINA: Voice Request: "NAINA, analyze my GitHub PRs and run unit tests."
    NAINA->>Memory: Query user context & previous project preferences
    Memory-->>NAINA: Return active project path & credentials metadata
    NAINA->>NAINA: Synthesize plan & generate Task Execution Contract
    NAINA-->>User: Voice Feedback: "I'm on it. Handing off system tests to CENANI."

    NAINA->>Router: Dispatch Task Contract (Intent: EXECUTE_TESTS)
    Router->>CENANI: Handshake & Transfer Contract (Payload JSON)

    CENANI->>CENANI: Parse steps: 1. Fetch PRs via GitHub API, 2. Run Pytest in Docker

    rect rgb(30, 41, 59)
        Note over CENANI,Target: CENANI Autonomous System Execution
        CENANI->>MCP: Call MCP Tool: `github_list_prs`
        MCP->>Target: GitHub API Request
        Target-->>MCP: Return PR list JSON
        MCP-->>CENANI: PR list payload

        CENANI->>MCP: Call MCP Tool: `docker_run_cmd` (pytest)
        MCP->>Target: Execute inside container
        Target-->>MCP: Test Execution Log (0 errors, 14 passed)
        MCP-->>CENANI: Command Output Stream
    end

    CENANI->>Router: Execution Result Contract (Status: SUCCESS, Logs)
    Router->>NAINA: Return Execution Result Payload
    NAINA->>Memory: Record execution outcome to Obsidian Vault & Vector DB
    NAINA-->>User: Voice Feedback: "All 14 unit tests passed successfully on your branch!"
```

6. Comprehensive Tech Stack Architecture

NAINA OS utilizes a modular, battle-tested modern technology stack:

```

+-----+
|                                     |
|               NAINA OS TECHNICAL STACK               |
|                                     |
+-----+
| Desktop Shell   | Tauri v2 (Rust) + Next.js 14 (App Router) + React 18 |
+-----+
| Styling & Motion | Tailwind CSS v4 + Framer Motion + Lucide Icons      |
+-----+
| Core Microservice| Python 3.11+ / FastAPI (AsyncIO, Pydantic v2)       |
+-----+
| High-Speed IPC  | Node.js v20+ / WebSockets / gRPC / Named Pipes      |
+-----+
| Local LLM Server | Ollama / llama.cpp (Qwen 2.5 7B/14B/72B, Qwen-VL)   |
+-----+
| Voice Engines   | STT: Faster-Whisper (CUDA/Metal)                   |
|                 | TTS: Piper TTS (Fast) / Kokoro-82M (High-Quality)   |
+-----+
| Relational & Vec | PostgreSQL 16 + pgvector (HNSW indexing)            |
+-----+
| Ephemeral Vector | ChromaDB / Qdrant (Local file storage / embedded)     |
+-----+
| Document Memory | Obsidian Vault Sync Daemon (Local Markdown .md)      |
+-----+
| Tool Execution  | Model Context Protocol (MCP) Host + Python SDK       |
+-----+
| Automation Engines| Playwright (Browser), ADB (Android), Docker Py SDK   |
+-----+

```

7. Subsystem Deep-Dives & Interfaces

7.1 Codebase Folder Structure

```

naina-os/
+-- apps/
|   +-- desktop/                # Tauri v2 Desktop Wrapper (Rust)
|   |   +-- src-tauri/          # Rust Core Engine & System Tray
|   |   +-- src/                # Next.js 14 Frontend UI
|   +-- mobile/                 # Android WebView / React Native Host
+-- packages/
|   +-- event-bus/              # Shared Typed Event Definitions (TS/Python)
|   +-- mcp-core/               # Model Context Protocol Client & Server SDK
|   +-- obsidian-bridge/        # Markdown Vault Parsing & Sync Engine
+-- services/
|   +-- kernel/                 # FastAPI Core Micro-Kernel & Router
|   |   +-- src/
|   |   |   +-- api/            # REST & WebSocket Endpoints
|   |   |   +-- core/           # System Event Bus & Security Sandbox
|   |   |   +-- personas/       # NAINA & CENANI Cognitive Engines
|   |   |   +-- memory/         # pgvector & ChromaDB Orchestrator
|   |   |   +-- router/         # LLM Model Cascade Engine
|   +-- voice-service/          # Real-Time Voice Processing Daemon
|   |   +-- stt_whisper.py       # Faster-Whisper Streaming Worker
|   |   +-- tts_kokoro.py        # Kokoro-82M Synthesizer Engine
|   +-- agents/                 # Specialized System Agents
|   |   +-- desktop_agent.py     # Win32 / POSIX GUI Automation
|   |   +-- android_agent.py     # ADB Bridge Controller
|   |   +-- browser_agent.py     # Playwright Automation Engine
|   |   +-- docker_agent.py      # Container Lifecycle Engine
+-- config/                     # System Configurations & Schemas
+-- docker/                     # Local Microservice Sandbox Configurations

```

7.2 Core Interfaces & Data Contracts

7.2.1 Event Bus Event Signature (TypeScript / Python Specification)

```
// Shared Interface across TS Frontend and Python Microservice
export type EventPriority = 'CRITICAL' | 'HIGH' | 'NORMAL' | 'LOW';

export interface NAINAEvent<T = Record<string, unknown>> {
  eventId: string;           // UUIDv4
  timestamp: string;         // ISO-8601 UTC string
  eventType: string;         // e.g., 'EVT_INTENT_DISPATCHED'
  sourceModule: string;      // e.g., 'interface.voice'
  targetModule: string;      // e.g., 'cognition.naina'
  priority: EventPriority;
  correlationId: string;     // Tracing ID across sequence flows
  payload: T;
  securityContext: {
    originUserId: string;
    capabilityToken: string;
    isSandboxApproved: boolean;
  };
};
```

```
# Python FastAPI Kernel Event Model
from pydantic import BaseModel, Field
from typing import Dict, Any, Optional
from enum import Enum
import uuid
from datetime import datetime

class EventPriority(str, Enum):
    CRITICAL = "CRITICAL"
    HIGH = "HIGH"
    NORMAL = "NORMAL"
    LOW = "LOW"

class SystemEvent(BaseModel):
    event_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    timestamp: str = Field(default_factory=lambda: datetime.utcnow().isoformat())
    event_type: str
    source_module: str
    target_module: str
    priority: EventPriority = EventPriority.NORMAL
    correlation_id: str
    payload: Dict[str, Any]
    capability_token: str
```

7.2.2 Model Context Protocol (MCP) Tool Execution Contract

```
class MCPToolCallRequest(BaseModel):
    call_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    server_name: str # e.g., "docker-sandbox"
    tool_name: str # e.g., "run_container"
    arguments: Dict[str, Any] # e.g., {"image": "python:3.11", "command": "pytest"}
    assigned_persona: str = "CENANI" # Must be CENANI
    timeout_seconds: int = 60

class MCPToolCallResponse(BaseModel):
    call_id: str
    status: str # "SUCCESS" | "FAILURE" | "DENIED"
    output: Any
    execution_time_ms: float
    error_message: Optional[str] = None
```

8. Memory Architecture & Knowledge Graph

NAINA OS employs a hybrid Tri-Tier Memory Engine designed to ensure infinite retention, instant vector retrieval, and absolute data portability.

```
flowchart LR
    subgraph Inputs
        InputMessage[New Experience / Context]
    end

    subgraph Memory_Engine [Layer 6: Multi-Tier Memory Engine]
        InputMessage --> Ephemeral[Tier 1: Ephemeral Working Memory\nIn-Memory Buffer]

        Ephemeral -->|Session Summary| RelationalVec[Tier 2: Relational & Vector Store\nPostgre]

        RelationalVec -->|HNSW Vector Embedding| VectorSearch[(Vector Index Search)]
        RelationalVec -->|Entity Extraction| KnowledgeGraph[(Graph Triples Node Store)]

        RelationalVec <-->|Real-Time Vault Sync| ObsidianStore[Tier 3: Obsidian Vault Mirror\nP]
    end

    subgraph Memory_Retrieval [Retrieval Cycle]
        Query[Intent Query] --> RouterMem{Retrieval Router}
        RouterMem -->|Semantic Match| VectorSearch
        RouterMem -->|Structured Knowledge| KnowledgeGraph
        RouterMem -->|Human Audit / Edit| ObsidianStore
    end
```

8.1 Memory Tiers

1. Tier 1: Ephemeral Working Memory: High-speed, in-memory sliding context window (Redis / RAM buffer) retaining current conversation turns and active tool states.
2. Tier 2: Structured Relational & Vector Store:
 - * PostgreSQL + pgvector: Stores structured session records, agent logs, and 1536-dimensional vector embeddings with HNSW indexing for sub-10ms cosine similarity retrieval.
 - * Knowledge Graph: Stores named entities and semantic relationships (e.g., (User)-[MAINTAINS]->(ProjectX)).
3. Tier 3: Obsidian Vault Mirror:
 - * Every semantic memory node is mirrored into a formatted .md file inside the user's local Obsidian Vault (/NainaMemory/).
 - * Bidirectional markdown links ([[ProjectX]], [[User_Preferences]]) reflect the underlying knowledge graph structure.

9. Security Architecture & Sandboxing

NAINA OS enforces a zero-trust model between agents and host operating system resources.

```
+-----+
|                                     |
|             SECURITY & SANDBOX ARCHITECTURE             |
|-----+
| [Layer 5] CENANI Agent Execution Request                  |
|   |                                                       |
|   v                                                       |
| [Layer 9] Capability Token & RBAC Inspector              |
|   |                                                       |
|   +--> Is Action Destructive? (File deletion, sudo, system config) |
|   |                                                       | |
|   |   +---[ YES ]--> Trigger Interactive Approval (`ask_permission`)|
|   |   |                                                       |
|   |   |   +---[ APPROVED ]--> Proceed                        |
|   |   |   +---[ REJECTED ]--> Abort Execution                |
|   |   |                                                       |
|   |   +---[ NO ]--> Auto-Grant Execution                    |
|   |                                                       |
|   v                                                       |
| Docker / Air-Gap Isolated Container Execution           |
|   |                                                       |
|   v                                                       |
| Immutable System Audit Log Record (PostgreSQL & Disk Log) |
+-----+
```

9.1 Capability Matrix & Permission Escalation

- * Level 0 (Read-Only): File reading in designated scratch space, public web scraping, local embedding generation. Executed automatically.
- * Level 1 (User Space Mutation): File creation, writing to workspace directories, sending notifications. Executed automatically with notification log.
- * Level 2 (System Mutation / Network): Package installation, executing arbitrary shell scripts, modifying Android device settings. Requires explicit interactive prompt via the ask_permission UI modal.
- * Level 3 (High-Risk Operations): sudo execution, database deletion, partition modification. Require explicit biometric / master passcode re-authentication.

10. Performance Benchmarks & Targets

To ensure a seamless digital human experience, NAINA OS establishes strict latency budgets across all pipeline stages:

Subsystem Component	Target SLA	Maximum Allowable Threshold	Optimization Technique
---	---	---	---
Voice Activity Detection (VAD)	< 30 ms	50 ms	Silero VAD C++ binding
Speech-to-Text (STT)	< 200 ms	350 ms	Faster-Whisper int8 quantization + CUDA
LLM Router Classification	< 50 ms	100 ms	Small 1B classifier model / Regex heuris
Local LLM Time to First Token (TTF)	< 300 ms	600 ms	Qwen-2.5 7B GGUF Q4_K_M on Metal/CUDA
Text-to-Speech (TTS) Stream Start	< 120 ms	250 ms	Kokoro-82M streaming audio chunks
End-to-End Voice-to-Voice Latency	< 700 ms	1200 ms	Parallelized streaming pipeline
pgvector Cosine Search (100k items)	< 8 ms	20 ms	HNSW Index (m=16, ef_construction=64)
MCP Tool Execution Overhead	< 15 ms	40 ms	Persistent async WebSocket IPC

11. Configuration Specifications

11.1 Main System Configuration (`naina\_os.config.json`)

```
{
  "system": {
    "version": "1.0.0",
    "environment": "production",
    "local_first_strict_mode": true,
    "log_level": "INFO"
  },
  "personas": {
    "naina": {
      "enabled": true,
      "voice_model": "kokoro-v1.0-empathetic",
      "temperature": 0.7,
      "base_model": "ollama/qwen2.5:14b-instruct-q4_K_M"
    },
    "cenani": {
      "enabled": true,
      "voice_model": "piper-en_US-lessac-medium",
      "temperature": 0.1,
      "base_model": "ollama/qwen2.5-coder:14b-instruct-q4_K_M"
    }
  },
  "memory": {
    "postgres_dsn": "postgresql://naina_kernel:secure_pass@localhost:5432/naina_db",
    "vector_dimensions": 1536,
    "obsidian_vault_path": "C:/Users/jonat/Documents/NainaSecondBrain",
    "auto_sync_interval_seconds": 30
  },
  "mcp_servers": {
    "desktop-automation": {
      "command": "python",
      "args": ["-m", "naina.agents.desktop"],
      "env": { "ENABLE_WIN32_BRIDGE": "1" }
    },
    "browser-playwright": {
      "command": "node",
      "args": ["packages/mcp-core/dist/browser_server.js"],
      "env": { "HEADLESS": "true" }
    }
  }
}
```

12. Deployment & DevOps Architecture

12.1 Local Docker Composition (`docker-compose.yml`)

```
version: '3.8'

services:
  naina-db:
    image: pgvector/pgvector:pg16
    container_name: naina-postgres
    restart: always
    environment:
      POSTGRES_DB: naina_db
      POSTGRES_USER: naina_kernel
      POSTGRES_PASSWORD: secure_pass
    ports:
      - "5432:5432"
    volumes:
      - naina_pgdata:/var/lib/postgresql/data

  ollama-engine:
    image: ollama/ollama:latest
    container_name: naina-ollama
    restart: always
    ports:
      - "11434:11434"
    volumes:
      - ollama_models:/root/.ollama
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]

  naina-kernel:
    build:
      context: .
      dockerfile: services/kernel/Dockerfile
    container_name: naina-kernel-core
    restart: always
    depends_on:
      - naina-db
      - ollama-engine
    ports:
      - "8000:8000"
    environment:
      - POSTGRES_DSN=postgresql://naina_kernel:secure_pass@naina-db:5432/naina_db
      - OLLAMA_HOST=http://ollama-engine:11434
    volumes:
      - ${HOME}/Documents/NainaSecondBrain:/vault

volumes:
  naina_pgdata:
  ollama_models:
```

13. Future Roadmap & Expansion

```
gantt
    title NAINA OS Development & Engineering Roadmap
    dateFormat YYYY-MM-DD
    section Phase 1: Core Kernel
    Micro-Kernel & Event Bus Architecture      :done,    p1a, 2026-01-01, 2026-03-31
    Dual Cognition Core (NAINA & CENANI)       :done,    p1b, 2026-03-01, 2026-05-31
    pgvector & Obsidian Vault Sync Engine      :active,  p1c, 2026-05-01, 2026-08-31

    section Phase 2: Multimodal & MCP
    Real-Time Streaming Voice Pipeline (Kokoro) :        p2a, 2026-08-01, 2026-10-31
    MCP Execution Host & Sandbox Security       :        p2b, 2026-09-01, 2026-11-30
    Playwright & Win32 Automation Bridge       :        p2c, 2026-10-15, 2026-12-31

    section Phase 3: Mobile & Multi-Agent
    ADB Android OS Controller Subsystem        :        p3a, 2027-01-01, 2027-03-31
    Multi-Agent Mesh & Distributed Consensus    :        p3b, 2027-03-01, 2027-06-30

    section Phase 4: Full Digital Human
    3D Avatar WebGL / Unreal Engine Rendering  :        p4a, 2027-06-01, 2027-12-31
```

14. Glossary of Terms & Acronyms

- * ADB: Android Debug Bridge; protocol used to send automation commands to Android devices.
- * Capability Token: Cryptographic session token scoping what tools an agent is authorized to invoke.
- * CENANI: The system-focused, logical, zero-emotion execution persona of NAINA OS.
- * Digital Human: An AI entity combining real-time voice, emotional intelligence, persistent memory, and multimodal vision to simulate human presence.
- * HNSW: Hierarchical Navigable Small World; graph-based algorithm for ultra-fast approximate nearest neighbor vector search.
- * MCP: Model Context Protocol; open standard created by Anthropic for connecting AI models to external tools and data sources.
- * NAINA: The primary, high-EQ digital human companion, planner, and memory keeper persona of NAINA OS.
- * Obsidian First: Architectural principle mandating that persistent knowledge must sync to human-readable Markdown files.
- * pgvector: Open-source vector similarity search extension for PostgreSQL.
- * Qwen-2.5 / Qwen-VL: State-of-the-art open-weights foundation LLM and vision-language models developed by Alibaba Cloud.
- * Tauri: Framework for building lightweight cross-platform desktop applications using native WebViews and Rust backends.
- * VAD: Voice Activity Detection; technology that detects when human speech begins and ends in an audio stream.

End of NAINA OS System Architecture & Engineering Specification -- Document NOS-ENG-SPEC-2026-V1.0